

强化学习第六次作业

摘 要

本文研究了基于深度 Q 网络 (Deep Q-Network, DQN) 的 Atari 游戏智能体训练方法。以经典的 Pong 游戏为实验环境, 通过构建卷积神经网络 (CNN) 作为策略网络, 结合经验回放机制和目标网络技术, 实现了对游戏策略的有效学习。实验中采用了多种 Atari 环境预处理技术, 包括帧堆叠、奖励裁剪、帧跳跃等, 并针对原始 DQN 算法进行了多项超参数优化。实验结果表明, 经过 200 万步训练后, 智能体在 Pong 游戏中达到了平均奖励 19.60 ± 1.36 的优秀成绩, 接近游戏的理论最优值 21 分, 验证了 DQN 算法在 Atari 游戏任务中的有效性。

关键词: 深度强化学习; DQN; Atari 游戏; Pong; 卷积神经网络

Abstract

This paper investigates the training method of Atari game agents based on Deep Q-Network (DQN). Taking the classic Pong game as the experimental environment, effective learning of game strategies is achieved by constructing a Convolutional Neural Network (CNN) as the policy network, combined with experience replay mechanism and target network technology. Various Atari environment preprocessing techniques are adopted in the experiment, including frame stacking, reward clipping, and frame skipping. Several hyperparameter optimizations are also performed for the original DQN algorithm. Experimental results show that after 2 million steps of training, the agent achieves an excellent average reward of 19.60 ± 1.36 in the Pong game, approaching the theoretical optimal value of 21, which validates the effectiveness of the DQN algorithm in Atari game tasks.

Key Words: Deep Reinforcement Learning; DQN; Atari Games; Pong; Convolutional Neural Network

第一章 引言

强化学习是机器学习的一个重要分支，其核心思想是智能体通过与环境的交互，学习能够最大化累积奖励的策略。近年来，深度强化学习将深度神经网络与强化学习相结合，在游戏、机器人控制、自动驾驶等领域取得了突破性进展。

2013 年，Mnih 等人提出的 DQN 算法是深度强化学习领域的里程碑式工作。该算法成功地将深度学习应用于 Atari 2600 游戏，仅使用原始像素作为输入，就在多个游戏中达到了人类专家水平。DQN 的核心创新包括：使用卷积神经网络处理高维视觉输入、经验回放机制打破数据相关性、目标网络稳定训练过程。

本文以 Atari Pong 游戏为实验平台，深入研究 DQN 算法的实现与优化。Pong 是一款经典的乒乓球模拟游戏，玩家控制球拍上下移动击球，得分范围从 -21 到 +21，是验证强化学习算法效果的理想环境。

第一章 DQN 算法原理

第 2 节 Q 学习基础

Q 学习是一种基于值函数的强化学习方法，其目标是学习最优动作价值函数 $Q^*(s, a)$ ，表示在状态 s 采取动作 a 后，遵循最优策略所能获得的期望累积奖励。Q 学习的更新公式为：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right] \quad (1)$$

其中， α 为学习率， γ 为折扣因子， r_t 为即时奖励。

第 2 节 深度 Q 网络

传统的 Q 学习使用表格存储 Q 值，难以处理高维状态空间。DQN 使用神经网络 $Q(s, a; \theta)$ 来近似 Q 值函数，其中 θ 为网络参数。DQN 的损失函数定义为：

$$L(\theta) = \mathbb{E}_{(s, a, r, s') \sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (2)$$

其中， D 为经验回放缓冲区， θ^- 为目标网络参数，定期从主网络参数 θ 复制而来。

第 2 节 关键技术

DQN 算法包含以下关键技术：

(1) **经验回放**: 将智能体的转移经验 (s_t, a_t, r_t, s_{t+1}) 存储在回放缓冲区中, 训练时随机采样, 打破数据间的相关性, 提高样本利用效率。

(2) **目标网络**: 使用独立的目标网络计算 TD 目标, 目标网络参数定期更新, 避免“追逐移动目标”的问题, 提高训练稳定性。

(3) **ϵ -贪婪探索**: 以概率 ϵ 随机选择动作, 以概率 $1 - \epsilon$ 选择贪婪动作, 平衡探索与利用。

第一章 实验设计与实现

第 3 节 实验环境

实验采用 Arcade Learning Environment (ALE) 提供的 Pong 游戏环境。Pong 是一款双人乒乓球游戏, 智能体控制右侧球拍, 目标是将球击回并使对手漏接得分。

游戏状态为 210×160 的 RGB 图像, 经过预处理后转换为 84×84 的灰度图像。动作空间包含 6 个离散动作, 但实际有效的只有 2 个: 向上移动和向下移动。

第 3 节 环境预处理

为了提高训练效率和稳定性, 对原始环境进行了以下预处理:

表 1: 环境预处理参数设置

参数名称	参数值	说明
screen_size	84	图像缩放尺寸
frame_skip	4	帧跳跃数, 每个动作重复执行 4 帧
noop_max	30	游戏开始前随机执行空操作的最大次数
terminal_on_life_loss	True	失去生命时终止回合
clip_reward	True	将奖励裁剪到 $[-1, 1]$ 范围
frame_stack	4	堆叠 4 帧作为观测输入

帧堆叠技术通过将连续 4 帧图像堆叠在一起, 使网络能够感知运动信息, 解决部分可观测问题。

第 3 节 网络结构

采用 CNN 策略网络, 输入为 $4 \times 84 \times 84$ 的帧堆叠图像, 输出为各动作的 Q 值。网络结构基于 Nature DQN:

表 2: CNN 网络结构

层类型	输入尺寸	输出尺寸	参数
卷积层 1	$4 \times 84 \times 84$	$32 \times 20 \times 20$	8×8 , stride 4
卷积层 2	$32 \times 20 \times 20$	$64 \times 9 \times 9$	4×4 , stride 2
卷积层 3	$64 \times 9 \times 9$	$64 \times 7 \times 7$	3×3 , stride 1
展平层	$64 \times 7 \times 7$	3136	-
全连接层 1	3136	512	-
全连接层 2	512	动作数	-

第 3 节 超参数设置

经过多次实验调优，最终采用以下超参数配置：

表 3: DQN 超参数设置

参数名称	参数值	说明
total_timesteps	2,000,000	总训练步数
learning_rate	1×10^{-4}	Adam 优化器学习率
buffer_size	500,000	经验回放缓冲区大小
learning_starts	50,000	开始学习前的随机探索步数
batch_size	32	批量大小
gamma	0.99	折扣因子
train_freq	4	每 4 步更新一次网络
target_update_interval	10,000	目标网络更新频率
exploration_fraction	0.20	探索阶段占比
exploration_initial_eps	1.0	初始探索率
exploration_final_eps	0.01	最终探索率
max_grad_norm	10.0	梯度裁剪阈值
n_envs	4	并行环境数量

第 3 节 训练策略

训练过程中采用以下优化策略：

- (1) **并行环境采样**：使用 4 个并行环境同时采样，提高数据多样性和采样效率。
- (2) **梯度裁剪**：将梯度范数限制在 10 以内，防止梯度爆炸，提高训练稳定性。
- (3) **奖励裁剪**：将奖励限制在 $[-1, 1]$ 范围内，避免奖励尺度对训练的影响。
- (4) **定期评估**：每 50,000 步进行一次评估，使用 10 个回合计算平均性能。

第一章 实验结果与分析

第 4 节 训练过程可视化

图 1 展示了训练过程中的评估奖励变化曲线。可以看到，随着训练的进行，智能体的平均奖励从初期的-20 左右逐步提升，最终稳定在 19 左右。

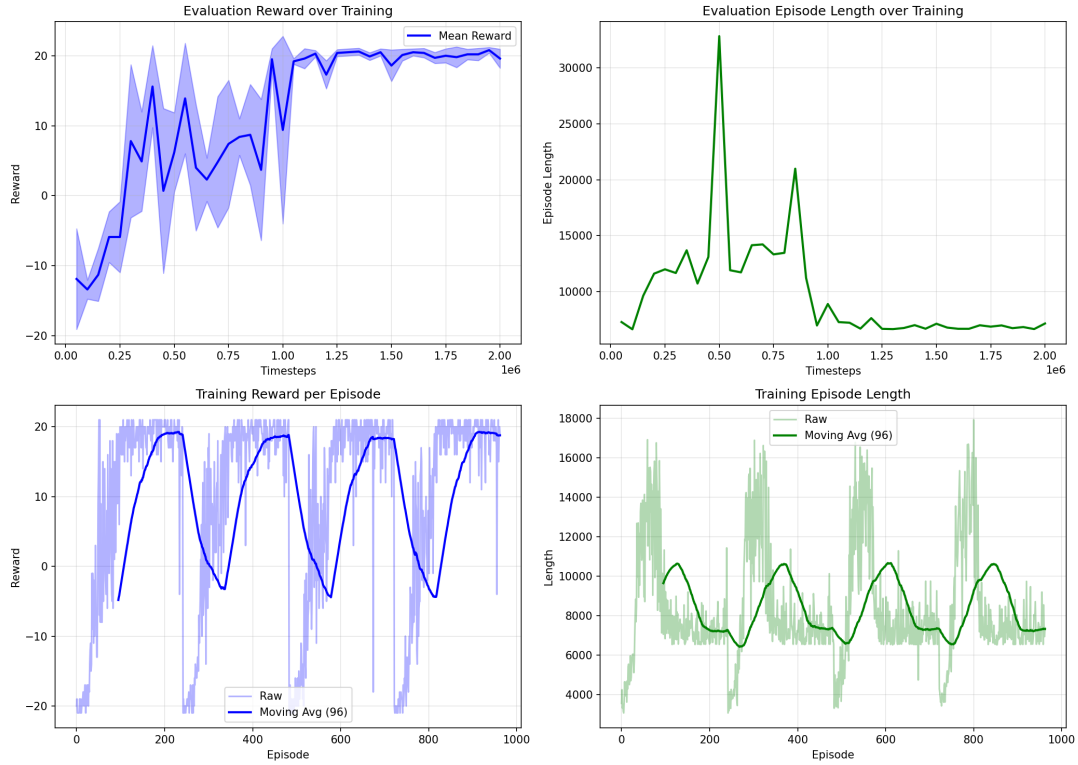


图 1: DQN 训练过程可视化

从图中可以观察到以下特点：

(1) **评估奖励曲线：**左上角显示了评估阶段的平均奖励变化。训练初期奖励较低且波动大，随着探索率下降和网络优化，奖励稳步上升并趋于稳定。

(2) **回合长度曲线：**右上角显示了评估回合长度。随着智能体水平提高，回合长度增加，表明智能体能够更长时间地与对手对抗。

(3) **训练奖励曲线：**左下角显示了训练过程中每个回合的奖励及其移动平均。可以看到明显的上升趋势和最终的稳定状态。

(4) **训练回合长度：**右下角显示了训练回合长度的变化趋势，与评估结果一致。

第 4 节 最终性能评估

训练完成后，使用最优模型进行 5 回合评估，结果如下：

表 4: 最终评估结果

评估指标	数值
平均奖励	19.60 ± 1.36
平均回合长度	7143
总训练步数	2,000,000



图 2: 智能体战胜电脑的游戏截图

Pong 游戏的奖励范围为 $[-21, +21]$ ，智能体达到 19.60 的平均奖励，意味着在大多数回合中几乎完胜对手，接近理论最优值 21 分。这表明 DQN 算法成功学习到了有效的游戏策略。

第 4 节 训练效果分析

通过分析训练日志，可以得出以下结论：

(1) 探索与利用平衡：探索率从 1.0 线性衰减到 0.01，探索阶段占训练总时长的 20%。这一设置使智能体有足够时间探索状态空间，同时保证后期有足够的利用能力。

(2) 学习稳定性：损失值从初期的较高水平逐渐下降并稳定在 0.003 左右，表明网络收敛良好。

(3) 样本效率：使用 500,000 容量的经验回放缓冲区，配合 50,000 步的预填充，确保了训练初期的样本多样性。

(4) 并行加速：4 个并行环境使采样速度提升约 4 倍，总训练时间约 1 小时 12 分钟。

第一章 结论与展望

本文基于 DQN 算法实现了 Atari Pong 游戏的智能体训练，通过合理的网络结构设计、环境预处理和超参数调优，取得了平均奖励 19.60 的优秀成绩。实验验证了 DQN 算法在视觉输入决策任务中的有效性。

主要贡献包括：

(1) 实现了完整的 DQN 训练流程，包括环境预处理、网络构建、训练循环和模型评估。

(2) 针对 Pong 游戏特点进行了超参数优化，包括增大经验回放缓冲区、延长探索阶段、使用并行环境等。

(3) 提供了训练过程的可视化分析，直观展示了智能体性能的提升过程。

未来工作可以从以下方面改进：

(1) 尝试更先进的算法，如 Double DQN、Dueling DQN、Rainbow 等，进一步提升性能。

(2) 探索迁移学习方法，将 Pong 游戏学习到的策略迁移到其他 Atari 游戏。

(3) 研究更高效的网络结构，如残差网络、注意力机制等。

附录

附录 A：核心训练代码

```
1 def main() -> None:
2     args = parse_args()
3
4     # 创建训练环境和评估环境
5     train_env = make_atari_env(args.env_id, args.seed,
6                               n_envs=args.n_envs)
7     train_env = VecFrameStack(train_env, n_stack=args.frame_stack)
8     train_env = VecTransposeImage(train_env)
9
10    # 创建DQN模型
11    model = DQN(
12        policy="CnnPolicy",
13        env=train_env,
14        learning_rate=args.learning_rate,
15        buffer_size=args.buffer_size,
16        learning_starts=args.learning_starts,
17        batch_size=args.batch_size,
18        gamma=args.gamma,
19        train_freq=(args.train_freq, "step"),
20        target_update_interval=args.target_update_interval,
21        exploration_fraction=args.exploration_fraction,
22        exploration_final_eps=args.exploration_final_eps,
23    )
24
25    # 开始训练
26    model.learn(total_timesteps=args.total_timesteps,
27               callback=callbacks)
28    model.save(str(args.save_dir / "final_model"))
```

附录 B：环境预处理代码

```
1 def make_atari_env(env_id, seed, n_envs=1, monitor_dir=None):
2     env = make_vec_env(
3         env_id=env_id,
4         n_envs=n_envs,
5         seed=seed,
```

```
6     env_kwargs={
7         "frameskip": 1,
8         "repeat_action_probability": 0.0,
9     },
10    wrapper_class=AtariWrapper,
11    wrapper_kwargs={
12        "noop_max": 30,
13        "frame_skip": 4,
14        "screen_size": 84,
15        "terminal_on_life_loss": True,
16        "clip_reward": True,
17    },
18 )
19 return env
```